Proyecto Cuatrimestral – Requerimientos Defensa Nº 5 (martes 20/05)

Modelado e implementación de una versión simplificada de un juego.

Caso de estudio: The Legend of Zelda: Link's Awakening.

Requerimientos para la entrega:

● **Diagrama reducido de clases**: generar un diagrama reducido de clases en UML (nombre de clases y relaciones en formato visual, únicamente), que permita modelar el proyecto propuesto. Se espera que, para esta instancia, la comisión se concentre en los aspectos lógicos, el tratamiento de datos para el parseo de niveles y las cuestiones gráficas. En este último caso, se espera una aproximación clara a la interconexión de las componentes lógicas de la aplicación con sus respectivas componentes gráficas. Será útil considerar específicamente cómo las entidades gráficas (capa vista) se encontrarán mapeadas una a una con las entidades lógicas (capa lógica) y cómo un cambio en una entidad lógica se verá reflejado en su respectiva entidad gráfica. Para esta instancia, todos los patrones de diseño requeridos por el enunciado deben estar definidos.

● **Diagrama extendido de clases**: generar un diagrama extendido de clases en UML (nombre de clase, sus atributos y operación), que permita modelar el proyecto propuesto. Se espera que, para esta instancia, la comisión se concentre en las operaciones que ofrecerán cada una de las entidades del modelo, prestando especial atención a la o las entidades que permitan la comunicación entre el motor del juego (lógica), el personaje principal, los power-ups y los enemigos cuando estos interactúan. Se espera que se comiencen a analizar las operaciones que permitirán la detección de colisiones entre entidades. Particularmente, se espera que se defina adecuadamente cómo se resuelve una colisión. Se espera que se comience a analizar el uso de hilos en el proyecto (con sus respectivas operaciones).

● **Diagramas de secuencia**: generar tres diagramas de secuencia por separado que permitan observar la cadena de mensajes intercambiados por las entidades correspondientes para implementar las siguientes interacciones:

1) Link colisiona con un power-Up y se aplica el efecto del mismo. Puede asumir que el primer evento de la cadena de mensajes se corresponde con la operación que indica la colisión, ya sea en la clase Link o en la clase Power-Up.

2) Link equipa su espada y realiza un ataque recargado. Indique la secuencia de mensajes enviados desde que la gráfica captura el evento asociado a la solicitud del ataque recargado. No es necesario considerar el chequeo de colisiones.

3) Un enemigo se desplaza automáticamente y su representación gráfica se actualiza en pantalla. Indique la secuencia de mensajes enviados desde que el hilo responsable de actualizar las entidades inicia un nuevo ciclo, hasta que el cambio queda reflejado en la interfaz. Considere que, en simultáneo, el hilo que lleva el tiempo descuenta un segundo al modo de juego vigente. No es necesario considerar el chequeo de colisiones.

Condiciones de entrega:

- Sobre la rama *principal* del repositorio privado de la comisión, dentro de la carpeta *Diagramas,* crear otra carpeta llamada *Diagramas de secuencia* y colocar allí los mismos (en lo posible, en formato PDF) **Respetar el nombre de la carpeta, sin excepción**.

Tips para la defensa:

● Concentrarse en qué elementos deben modelarse como clases y/o interfaces, qué características pueden factorizarse en entidades más generales, qué clases estarán parcial o totalmente implementadas y qué relaciones se definirán entre las entidades de la solución propuesta.

● Preste especial atención a entidades más abstractas del problema (tal vez no las encuentra notadas o descriptas en el enunciado del proyecto, pero son necesarias), tales como, por ejemplo, Juego, Mapa, Nivel, Ranking, etc.

● Efectivice el modelado de los patrones de diseño solicitados y de los que considere relevante incorporar.

● En cualquiera de las decisiones que tome respecto a la integración gráfica-lógica de las entidades, recuerde el modelo de 3 capas comentado en clase e intente mantener estas funcionalidades lo más separadas posible.